Module 1

Introduction to Web Development

Agenda

1. How Websites and Web Applications Work
2. Frontend vs Backend Development
3. Client-Server Architecture
4. Understanding Browsers and Web Servers
5. HTTP and URLs
6. Setting Up VS Code
7. Browser Developer Tools
8. Creating and Running Your First Web Page

© Copyright Intellipaat. All rights reserved.

How Websites and Web Applications Work

© Copyright Intellipaat. All rights reserved.

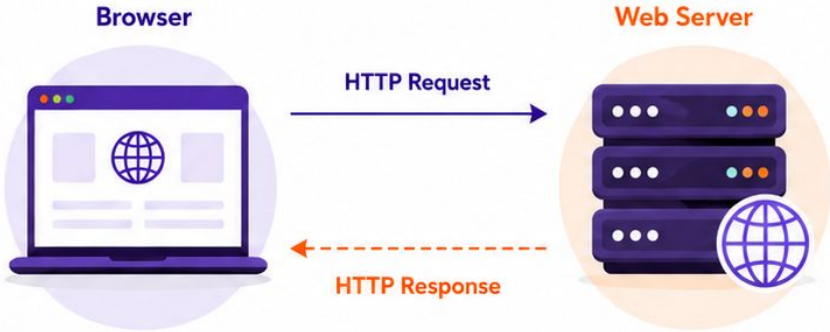
What Is a Website?



A **website** is a collection of related **web pages** and resources that can be accessed through the internet using a **web browser** and a **URL**.

How It Works

1. **User** — User wants to access a website.
2. **Web Browser** — Browser sends a request for the website.
3. **Website / Web Server** — Web server receives the request and processes it.
4. **Web Page** — Server finds the requested page and prepares the response.
5. **Displayed in Browser** — Browser receives the page and displays it to the user.



In short, the browser acts as a middleman that connects the user to the website and shows the requested content.

© Copyright Intellipaat. All rights reserved.

Web Page vs Website

Web Page

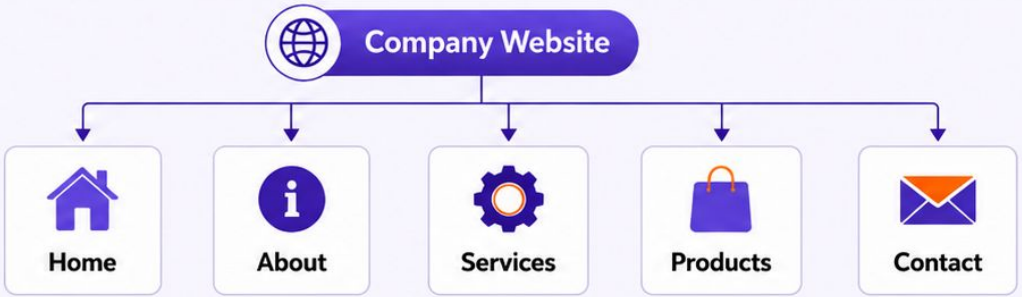
- A **single** page or document
- Has its own **URL**

Website

- A **collection** of related web pages
- Usually organized under a **common domain**

Example

A company **website** may contain several related **web pages**:

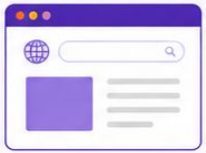


© Copyright Intellipaat. All rights reserved.

Web Browser and URL

What is a Web Browser?

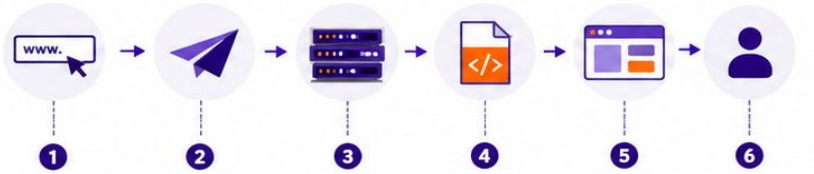
A **web browser** is a software application used to access, retrieve, and display websites and web pages from the internet.



Examples:

Google Chrome | Mozilla Firefox | Microsoft Edge

Browser Workflow

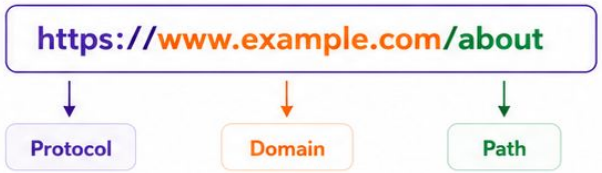


1. **Enter URL** — User enters the website address.
2. **Send Request** — Browser sends request to the web server.
3. **Receive Web Resources** — Server processes and sends the resources.
4. **Parse HTML + CSS + JavaScript** — Browser reads HTML, CSS, and JavaScript.
5. **Render Web Page** — Browser builds and displays the page.
6. **User Interacts** — User interacts with the web page.

What Is a URL?

URL (Uniform Resource Locator) is the address used to locate a resource on the web.

Basic URL Structure



- **HTTPS** → Communication protocol
- **www.example.com** → Domain name
- **/about** → Specific page/resource

© Copyright Intellipaat. All rights reserved.

How a Website Request Works

When a user enters a website address, the **browser sends a request** to a **web server**, and the server **sends the required resources** back to the browser.

Request–Response Process

1. **User enters URL**
   User types the website address in the browser.
2. **Browser sends HTTP request**
   Browser sends a request to the web server for the resources.
3. **Web server receives request**
   The server receives and understands the request.
4. **Server finds or generates resources**
   The server retrieves or generates the requested resources.
5. **Server sends HTTP response**
   Server sends the requested resources back to the browser.
6. **Browser displays the webpage**
   Browser processes the resources and displays the webpage to the user.

Example

When the user enters:

https://example.com

The browser requests the website's resources, such as:

HTML	CSS	JavaScript	Images
page structure	styling	behavior	and other assets



The browser then processes these **resources and renders the webpage.**

© Copyright Intellipaat. All rights reserved.

How Web Applications Work

A **web application** is an interactive application that runs through a web browser and allows users to **perform actions, enter data, and receive dynamic results**.

How It Works

1	**User**	User interacts with the web application.
2	**Web Browser**	Browser sends requests and displays results.
3	**Frontend**	Handles UI/UX and sends requests to the server.
4	**Backend Server**	Processes business logic and manages requests.
5	**Database**	Stores and retrieves application data.
6	**Backend processes data**	Server processes data from the database.
7	**Response sent to Browser**	Server sends a response back to the browser.
8	**User sees updated result**	Browser renders the result and displays it to the user.

Example: Online Shopping



When a user places an order:

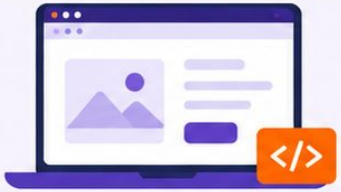
1. User selects a product.
2. Browser sends the order to the server.
3. Backend processes the order.
4. Database stores the order information.
5. Server sends a response.
6. Browser displays the order confirmation.

© Copyright Intellipaat. All rights reserved.

Frontend vs Backend Development

© Copyright Intellipaat. All rights reserved.

What Is Frontend Development?



Frontend development is the part of web development that focuses on everything the **user sees** and **interacts with** in a web browser.



Main Technologies

HTML → Defines the structure and content

CSS → Controls the appearance and layout

JavaScript → Adds interaction and dynamic behavior

Example: Online Shopping

For an **online shopping** website, the frontend includes:

- Product images and names
- Navigation menus
- Search bar
- Buy Now — Buttons
- Shopping cart interface
- Product pages

© Copyright Intellipaat. All rights reserved.

What Is Backend Development?

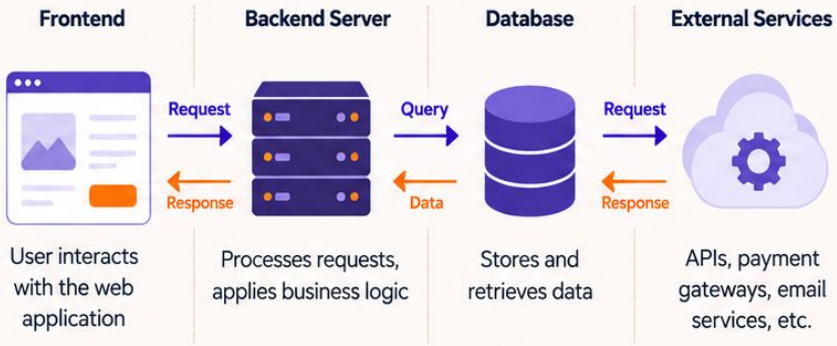


Backend development is the part of web development that runs on the **server** and handles the **logic, data, and operations** behind a website or web application.

What Does the Backend Do?

- **Processes requests** from the frontend
- Handles **application logic**
- Communicates with **databases**
- Provides **APIs** for data exchange
- Manages **authentication and authorization**
- Sends **results** back to the frontend

How the Backend Works



Key Responsibilities

Business Logic
Executes rules and workflows

Data Management
Create, read, update, and delete data

Security
Ensure secure access and data protection

Performance
Optimize responses and system efficiency

© Copyright Intellipaat. All rights reserved.

Frontend vs Backend

Frontend

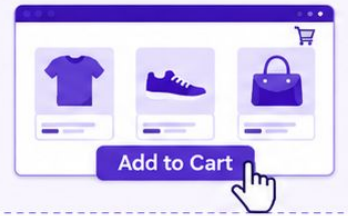
- Runs in the **browser**
- User **interface**
- **HTML, CSS, JavaScript**
- Displays and **collects information**
- Directly **visible to users**

Backend

- Runs on the **server**
- Application **logic**
- **Server-side** technologies
- Processes and **manages information**
- Mostly works **behind the scenes**

Example

In an online shopping application:



Frontend:
Displays products and provides the Add to Cart button.

Request

Response



Backend:
Processes the cart request and retrieves or updates the relevant data.

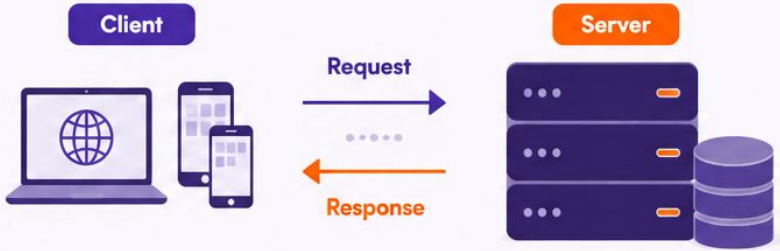
© Copyright Intellipaat. All rights reserved.

Client-Server Architecture

© Copyright Intellipaat. All rights reserved.

Client and Server

The **client-server model** is a basic architecture in which a client requests resources or services, and a server provides them.



What Is a Client?

A **client** is a device or application that requests and uses resources from a server.

Examples

- Web browser
- Smartphone
- Laptop

VS

What Is a Server?

A **server** is a computer or system that receives requests and provides resources or services to clients.

Examples

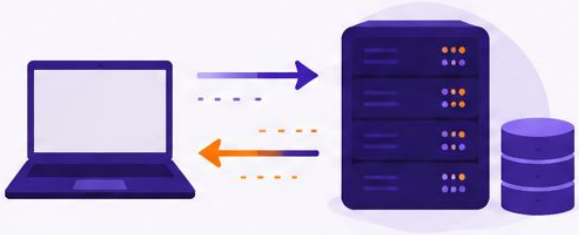
- Web server
- Application server
- Database server

© Copyright Intellipaat. All rights reserved.

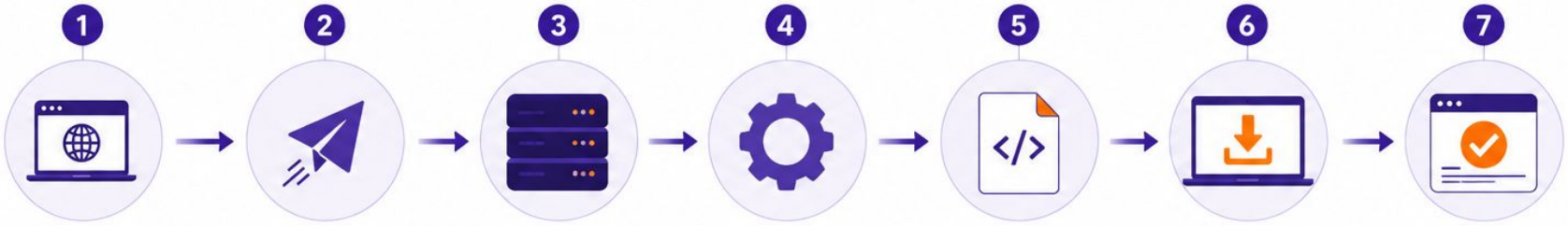
Client-Server Request Flow



The **client-server request flow** describes how a client communicates with a server to obtain a resource or perform an operation.



Request-Response Flow



Client
The client initiates a request.

Sends Request
The client sends a request to the server.

Server Receives Request
The server receives the request.

Server Processes Request
The server processes the request and prepares the data.

Server Sends Response
The server sends the response back to the client.

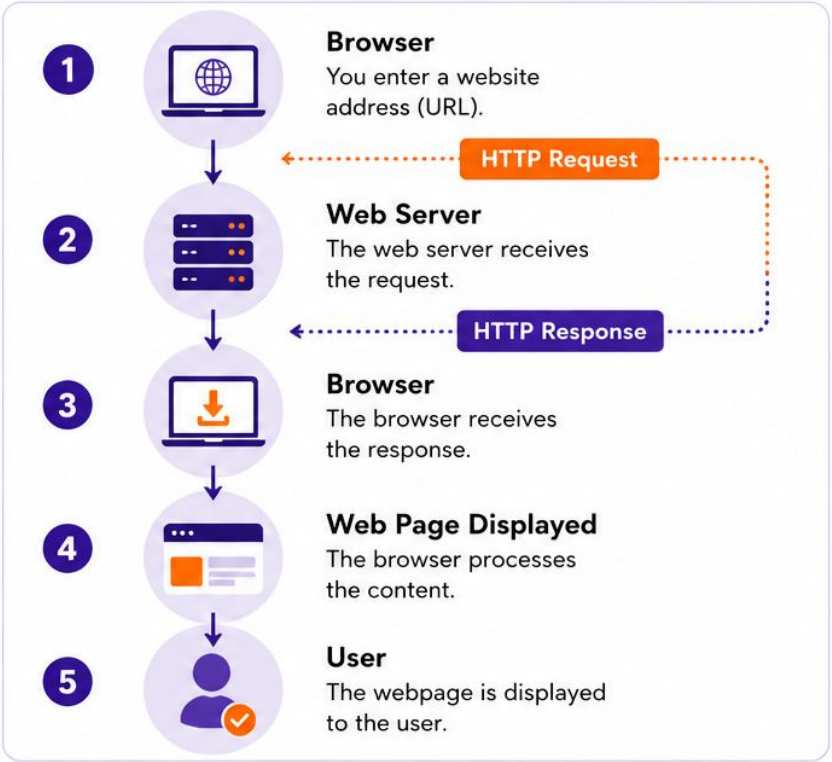
Client Receives Response
The client receives the response from the server.

Result Displayed to User
The result is displayed to the user.

© Copyright Intellipaat. All rights reserved.

Example — Opening a Website

This example shows the basic flow that happens when you open a website in a browser.



What Happens?

Client: Requests a webpage.

Server: Receives and processes the request.

Response: Server sends the requested resources or result.

Client: Processes the response and displays it to the user.

© Copyright Intellipaat. All rights reserved.

Understanding Browsers and Web Servers

© Copyright Intellipaat. All rights reserved.

What Does a Web Server Do?

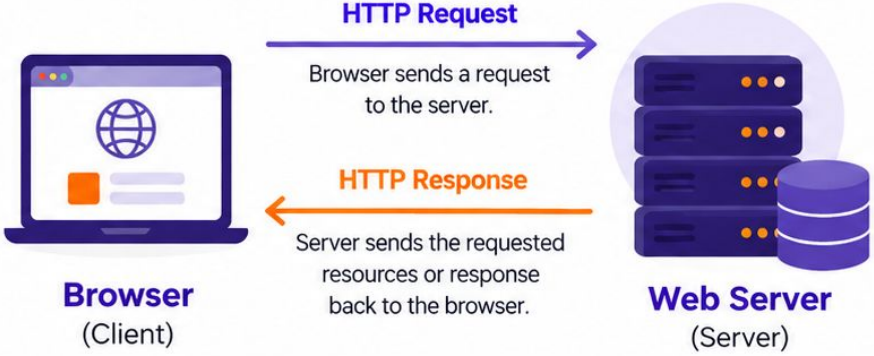


A **web server** is a system that receives requests from clients, processes them, and sends the requested web resources or responses back to the client.

Main Functions of a Web Server

- Receives **HTTP requests** from browsers
- Finds or generates **requested resources**
- **Processes requests** when required
- Sends **HTTP responses** back to the browser
- Serves resources such as **HTML, CSS, JavaScript,** and **images**

Browser ↔ Web Server



© Copyright Intellipaat. All rights reserved.

Example



When a user requests a **webpage**:

1. **Browser**
   User enters the request.
   "Give me /about"
2. **Web Server**
   The request reaches the web server.
3. **Finds /about page**
   The server locates the requested resource.
4. **Sends response**
   The server sends the page (response) back.
5. **Browser**
   The browser receives the response.
6. **Browser displays the page**
   The webpage is rendered and shown to the user.

What's Happening?

1. **Client:** The **browser** sends a request for the /about webpage.
2. **Server:** The **web server** receives the request and processes it.
3. **Server:** The **server** finds the /about page or generates it.
4. **Server:** The **server** sends the requested resource (response).
5. **Client:** The **browser** receives the response.
6. **Client:** The **browser** processes and displays the webpage to the user.

© Copyright Intellipaat. All rights reserved.

HTTP and URLs

© Copyright Intellipaat. All rights reserved.

What Is HTTP?

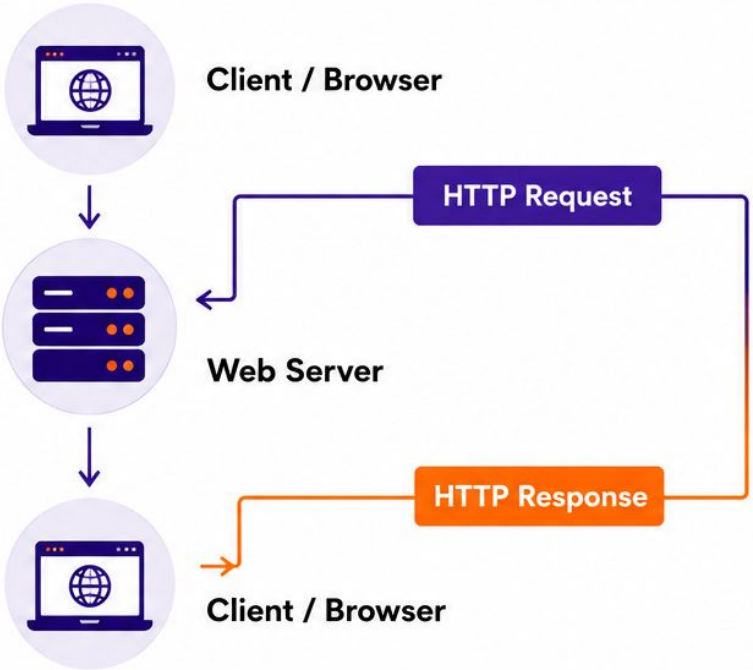
HTTP (HyperText Transfer Protocol) is a communication protocol used to transfer resources and information between a **client** and a **web server**.

Common HTTP Methods

GET → Requests data or a resource from the server.

POST → Sends data to the server, often to create or submit something.

HTTP Request and Response



© Copyright Intellipaat. All rights reserved.

HTTP Status Codes

The server uses **status codes** to indicate the result of a request.

Code	Meaning
200	Request successful
404	Resource not found
500	Server error



Example

When you open a webpage, the browser may send a **GET request** for the page. The server responds with the requested resource and a status code such as **200 OK.**

© Copyright Intellipaat. All rights reserved.

Understanding URLs



A **URL** (**Uniform Resource Locator**) is the address used to locate a resource on the web.

Example URL



https://example.com/products?id=10#details

Parts of a URL

https://example.com/products?id=10#details

Protocol	Domain	Path	Query	Fragment
Defines how the browser should access the resource.	The website name or address.	The specific location of the resource on the server.	Extra parameters sent to the server.	Points to a specific part within a page.
https://	example.com	/products	?id=10	#details

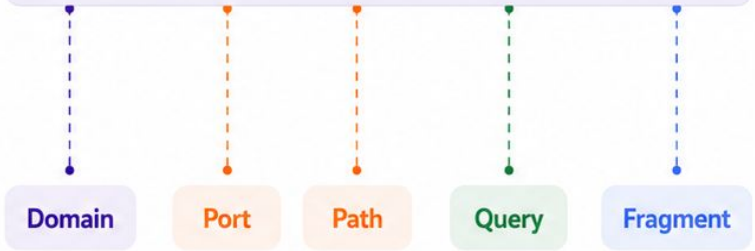
© Copyright Intellipaat. All rights reserved.

Understanding URLs

Part	Purpose	Example
Protocol	Defines how the resource is accessed	`https`
Domain	Identifies the website/server	`example .com`
Port	Identifies a specific network service	`:8080`
Path	Identifies a specific resource	`/products`
Query Parameters	Sends additional information	`?id=10`
Fragment	Points to a specific section of a resource	`#details`

Complete Example

`https://example.com:8080/products?id=10#details`



Note: The port is optional and is often omitted because standard ports are used automatically.

© Copyright Intellipaat. All rights reserved.

Setting Up VS Code

© Copyright Intellipaat. All rights reserved.

Installing VsCode

Linux

Run These Commands:

- sudo apt update
- sudo apt install software-properties-common apt-transport-https wget -y
- wget -q https://packages.microsoft.com/keys/microsoft.asc -O- | sudo apt-key add -
- sudo add-apt-repository "deb [arch=amd64] https://packages.microsoft.com/repos/vscode stable main"
- sudo apt update



Windows

Visit https://code.visualstudio.com/Download

Click "Download for Windows."

Then install it.



MacOS

Go to https://code.visualstudio.com/Download

Click download for mac OS and install it.



© Copyright Intellipaat. All rights reserved.

Setting Up Vs Code

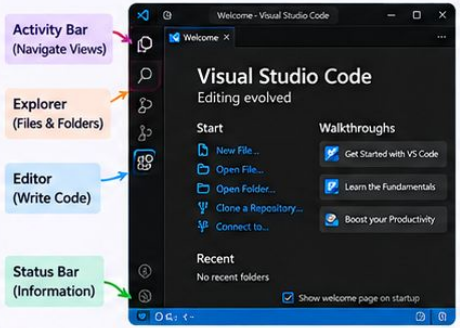
1 Download and Install VS Code

- Go to the official website:
  https://code.visualstudio.com
- Click on **Download** for your OS (Windows / macOS / Linux).
- Run the installer and complete the setup.



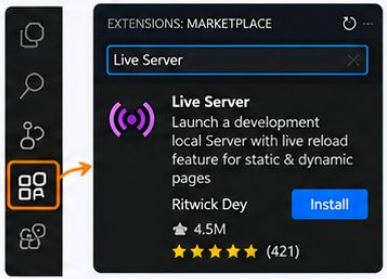
2 Understand the VS Code Interface

- Open VS Code. You will see the following interface.
- Familiarize yourself with the main parts:



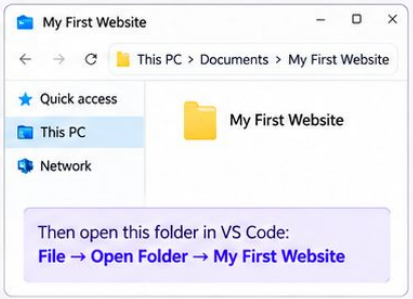
3 Install Useful Extensions

- Click the **Extensions** icon in the left sidebar (or press **Ctrl + Shift + X**).
- Search for useful extensions (e.g., **Live Server**).
- Click **Install**.



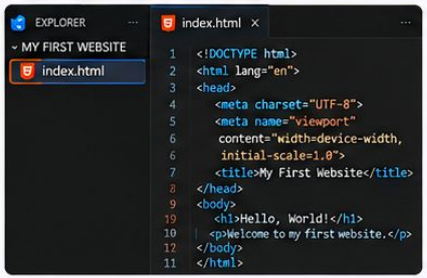
4 Create a Project Folder

- Create a folder on your computer for your project (e.g., My First Website).
- Open this folder in VS Code (File → **Open Folder**).



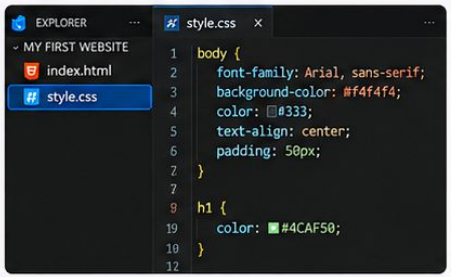
5 Create index.html

- In the Explorer panel, click the **New File** icon.
- Name the file **index.html**.
- Add the following HTML code.



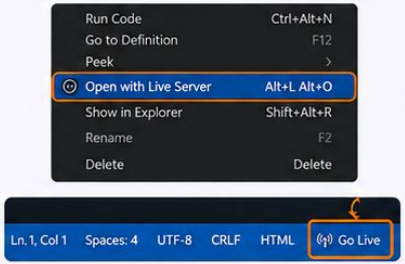
6 Create style.css

- Create another file in the same folder.
- Name the file **style.css**.
- Add the following CSS code.



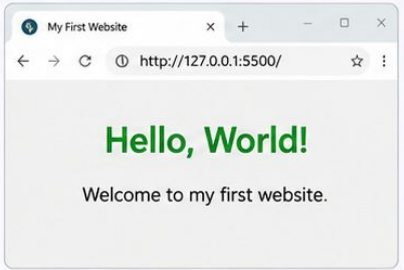
7 Start Live Server

- Open **index.html**.
- Right-click in the editor and select "**Open with Live Server**".
- or Click the **Go Live** button at the bottom-right.



8 View the Website

- The website will automatically open in your default browser.
- You will see a page like this:



© Copyright Intellipaat. All rights reserved.

Creating a Web Project

A **web project** is a folder that contains the files needed to build and manage a website.

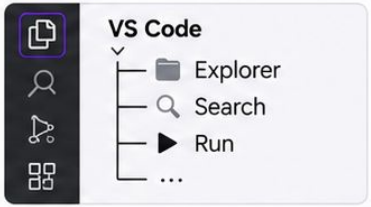
Step-by-Step

1 — Step 1 — Open VS Code

Launch **Visual Studio Code** on your computer.

2 — Step 2 — Open the Explorer

Click the Explorer icon on the left side of VS Code.



3 — Step 3 — Create/Open a Project Folder

Click Open Folder and create or select a folder named:

`my-website`

The folder will appear in the **Explorer** panel.

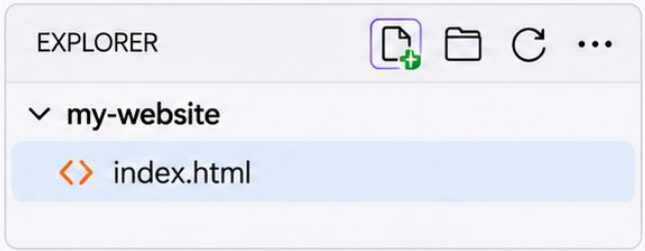
4 — Step 4 — Create an HTML File

In the Explorer panel, click the **New File** icon

Enter:

```
index.html
```

Then press **Enter.**



Tip: HTML files have the extension `.html` (e.g., `index.html`).

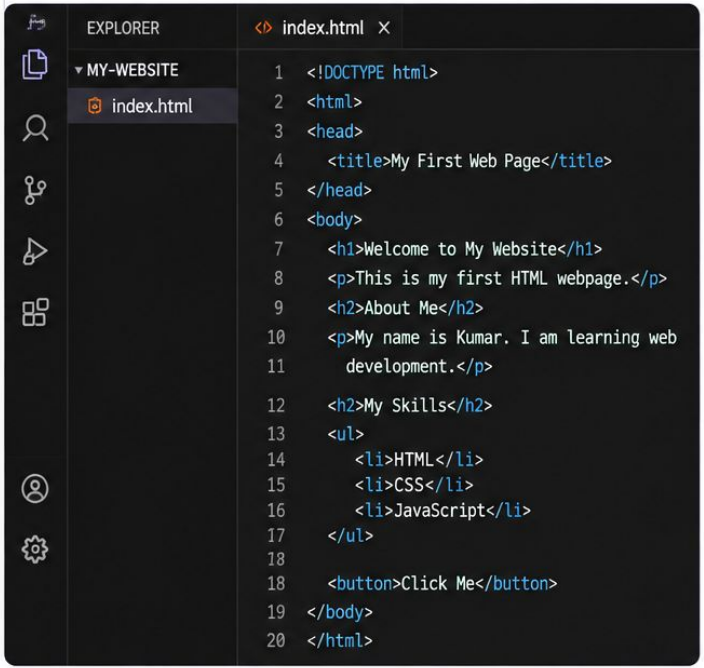
Once these steps are done,
you are ready to start building your website!

© Copyright Intellipaat. All rights reserved.

Creating a Web Project

5 Step 5 — Write HTML Code

Click `index.html` in the Explorer and write your HTML code in the editor.



6 Step 6 — Save the File

Save the file using:

- **Windows/Linux:** `Ctrl + S`
- **Mac:** `⌘ + S`

Final Project Structure



Image how it looks after writing this code



© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Set Up Vs Code and Create Your First Webpage

Objective

Install **Visual Studio Code** on your computer and configure it for basic web development.
After installing VS Code, complete the following steps:

1 Install VS Code

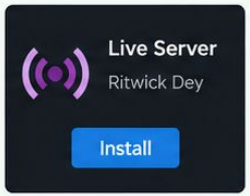
Download and install **Visual Studio Code** on your computer.



Visual Studio Code

2 Add Extension

Install the **Live Server** extension by **Ritwick Dey**.



3 Create Project Folder

Create a new project folder for your webpage.



my-website

4 Create an HTML File

Create an HTML file inside the project folder.



5 Write HTML Code

Write a basic HTML webpage that displays **"Hello World!"**.

```
index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Hello World</title>
6  </head>
7  <body>
7      <h1>Hello World!</h1>
9  </body>
10 </html>
```

6 Run the Webpage

Run the webpage in a browser using the installed development tools and verify that it works correctly.



© Copyright Intellipaat. All rights reserved.

Browser Developer Tools

© Copyright Intellipaat. All rights reserved.

Introduction to Developer Tools

Browser Developer Tools (DevTools) are built-in tools that help developers **inspect**, **test**, and **debug** web pages.

How to Open DevTools

Common methods:

- **Right-click → Inspect**
- **Chrome/Edge:** `F12` or `Ctrl + Shift + I`
- **Mac:** `⌘ + Option + I`

Important Panels

Panel	Purpose
Elements	Inspect and modify HTML/CSS
Console	View messages and JavaScript errors
Network	Monitor requests and responses
Sources	View and debug source files

DevTools helps you **understand** how your website works, **find issues**, and **improve performance.**

© Copyright Intellipaat. All rights reserved.

Inspecting and Debugging a Web Page

Inspecting allows developers to examine the HTML and CSS of a webpage, while **debugging** helps identify and fix problems.

Inspect HTML Elements

Using the **Elements** panel, developers can:

- View the HTML structure
- Select specific elements
- Temporarily modify HTML
- View and change CSS properties

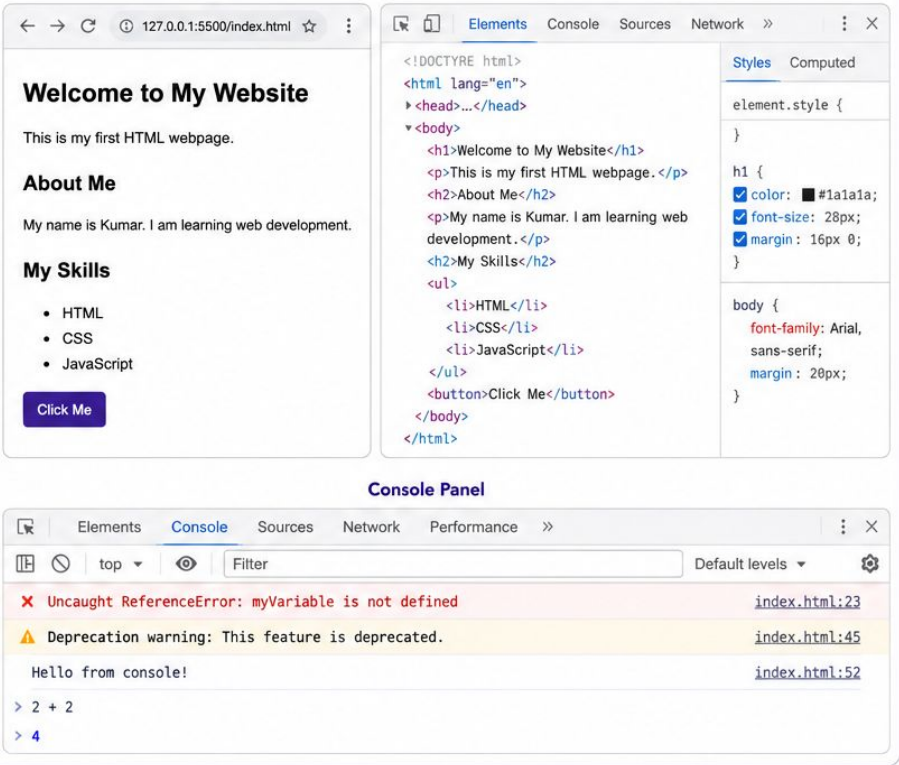
Changes made directly in DevTools are **temporary** and do not modify the original source file.

View Console Messages

The **Console** panel helps developers:

- View JavaScript errors
- Read warnings and messages
- Test JavaScript commands

Example: Chrome DevTools



© Copyright Intellipaat. All rights reserved.

Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.